Name:- Aaryan N. Raorane Div:- 5 Roll No.:- 5004
Batch:- A1 Branch:- AI&DS SAP ID:- 60018250070

Experiment No 14

Title:- SQLite, DDL and DML

Aim:- To establish a connection between Python and an SQLite database and perform Data Definition Language (DDL) and Data Manipulation Language (DML) operations programmatically.

Theory:-

Q1 What is SQLite? How does it differ from other database management systems such as MySQL or PostgreSQL?

⇒ SQLite is a small relational database engine that is built into the application as a library. It stores the complete database, with its tables and data, in a single file on the disk, and it is accessed using SQL commands. Python includes the sqlite3 module in its standard library, so a database can be created and used without installing anything extra.

Point	SQLite	MySQL	PostgreSQL
Architecture	Embedded library inside the application	client and server	client and server.

Sundaram
FOR EDUCATIONAL USE

Server Process	Not required	A server must be running	A server must be running.
Storage	one single file	Files managed by the server	Files managed by the server
Setup	No installation or configuration needed.	Needs installation and configuration	Needs installation and configuration.
Users and access	No user accounts or passwords	User accounts with row privileges	User accounts with roles and privileges.
Concurrent use	Best for one or a few writers at a time	Many users at the same time	Many users and heavy workloads.
Typical use	Small apps, mobile apps, learning testing	web applications	Large Business system.

Sundaram
FOR EDUCATIONAL USE

The first program connects Python to an sqlite database and performs the data Definition language operation CREATE TABLE, followed by the Data Manipulation language operations INSERT and SELECT

code:

```
import sqlite3

conn = sqlite3.connect("college.db")
cur = conn.cursor()

cur.execute("CREATE TABLE IF NOT EXISTS
            students(id INTEGER PRIMARY KEY,
                     name TEXT,
                     marks INTEGER)")

cur.execute("INSERT INTO students(name, marks)
            VALUES('Aarav', 82)")
cur.execute("INSERT INTO students(name, marks)
            VALUES('Riya', 91)")
conn.commit()

cur.execute("SELECT * FROM students")
for row in cur.fetchall():
    print(row)

conn.close()
```

Sundaram
FOR EDUCATIONAL USE

output:-

(1, 'Aarav', 82)
(2, 'Riya', 91)

The second program uses the DML operations UPDATE and DELETE, and the DDL operations ALTER TABLE and DROP TABLE. The commit() method saves the changes to the database file.

code:-

```
import sqlite3

conn = sqlite3.connect("college.db")
cur = conn.cursor()

cur.execute("UPDATE students SET marks=88
            WHERE name='Aarav'")
cur.execute("DELETE FROM students
            WHERE name='Riya'")
cur.execute("ALTER TABLE students
            ADD COLUMN city TEXT")
conn.commit()
cur.execute("SELECT * FROM students")
print(cur.fetchall())
cur.execute("DROP TABLE students")
conn.commit()
conn.close()
```

Sundaram
FOR EDUCATIONAL USE

output :-

[(1, 'Aarav', 88, None)]

Q> why is SQLite considered a serverless and lightweight database?

=> SQLite is serverless because it does not need a seperate database server process. The SQLite library runs inside the same process as the Python program and reads and writes the database file directly, so there is no network connection, no port and no login to set up. It is lightweight because the library is small, it needs no installation or configuration, it uses little memory, and the whole database is kept in one ordinary file that can be copied, moved or deleted like any other file.

code :-

```
import sqlite3
import os

conn = sqlite3.connect("light.db")
conn.execute("CREATE TABLE demo(id INTEGER)")
conn.commit()
conn.close()
```

Sundaram
FOR EDUCATIONAL USE

print("Database file exists: " + str(os.path.exists("light.db")))

print("size in bytes: " + str(os.path.getsize("light.db")))

output :-

Database file exists : True
Size in Bytes : 8192

The output shows that creating a database only produced one small file on the disk, with no server started at any point.

conclusion :-

A connection between Python and SQLite was made using the sqlite3 module, and tables were created, changed and removed using DDL commands. Records were inserted, updated, deleted and displayed using DML commands, with commit() saving the changes. SQLite needs no server and keeps the database in one file, which makes it a simple choice for small applications.

Sundaram
FOR EDUCATIONAL USE